

DineFlow — From Restaurant Website to Complete Ordering Platform

Most restaurant websites stop at displaying a menu. DineFlow connects customers, ordering workflows, payments, authentication, and restaurant administration in one system — built to find out what it actually takes to go from a polished frontend to a working business application with persistent data, server-side logic, and real operational workflows.

[View Live Site →](#)

[View Source \(GitHub\)](#)

ROLE

Full-Stack Developer (solo)

STACK

Next.js 16 · TypeScript · PostgreSQL

INTEGRATIONS

SSLCommerz · Resend · Supabase

STATUS

Live in production

17	9	4	0
API routes	database models	third-party integrations	known vulnerabilities

Try it yourself — this isn't a click-through prototype

Most portfolio demos only let you look at the customer side. The actual engineering in DineFlow lives in the admin dashboard — real order management, live payment reconciliation, analytics computed from real data — so there's a public login for it. No signup, no waiting for a response from me.

EMAIL

demo@dineflow.example

PASSWORD

TryMeOut2026

[Open Admin Login →](#)

✓ **Open:** full order pipeline, payment reconciliation, analytics, every page



Restricted: editing the menu, categories, settings, and image uploads — so the demo stays intact for the next visitor

OVERVIEW

What DineFlow is — and why it's more than a UI project

I built DineFlow to explore what it takes to move from a visually polished frontend to a functioning business application — persistent data, server-side logic, authentication, and real operational workflows, not just a UI that looks finished. It isn't a stand-in for a business idea I never tested: I designed the system, connected every layer myself, integrated three external services, and I'm the one who found and fixed the bugs before they reached a user — including one caught while writing the "Bugs Found" section of this very case study (see below).

Customers browse a menu, order, pay online or on delivery, and track their order status in real time. The restaurant's side manages the entire menu, watches orders arrive live, moves orders through a status pipeline, reconciles Cash on Delivery payments, and reads sales analytics — all behind real authentication, not a hidden link.

Customer Experience

Menu browsing with search & filters, cart, checkout with real online payment, live order tracking, order history behind login.

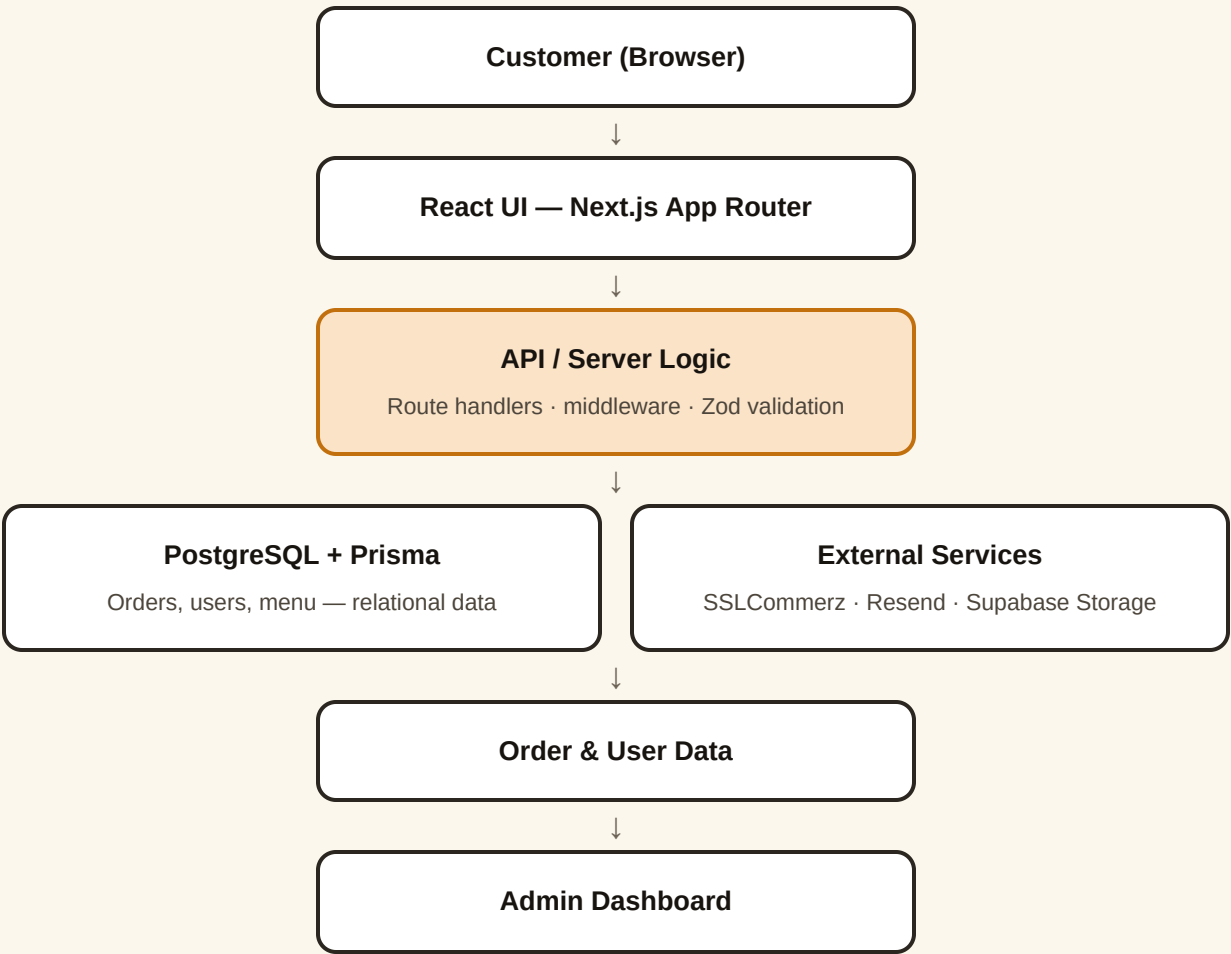
Admin Back Office

Role-protected dashboard: menu & category management with photo uploads, live order pipeline, payment reconciliation, sales analytics.

Real Infrastructure

Postgres database, session-based auth, a real payment gateway, real file storage, and real transactional email — every phase below is live.

How the pieces connect



Customer actions are processed through server-side logic before anything is written to persistent storage — the frontend never talks to the database directly. Prisma enforces structured relationships between users, menu items, categories, and orders, while every route handler validates the request shape and checks authorization before executing a mutation. External services are each isolated behind their own module, so a failure in one — an email provider outage, for instance — can never cascade into blocking a core business action like checkout.

PROJECT SCOPE

What DineFlow demonstrates, layer by layer

FRONTEND

- Responsive React interface (Next.js App Router)
- Context-based state management — cart, catalog, auth, orders, toasts
- Live-feeling UI: polling-based order tracking, real-time admin notifications
- Forms with client- and server-side validation

DATA

- PostgreSQL, hosted on Supabase
- Prisma ORM, 9 relational models
- Row Level Security as defense-in-depth
- Serverless-safe connection pooling strategy

BACKEND

- Server-side business logic across 17 API routes
- Role-based authentication & authorization (NextAuth)
- Input validation on every route (Zod)
- Rate limiting at the edge (Vercel Firewall)

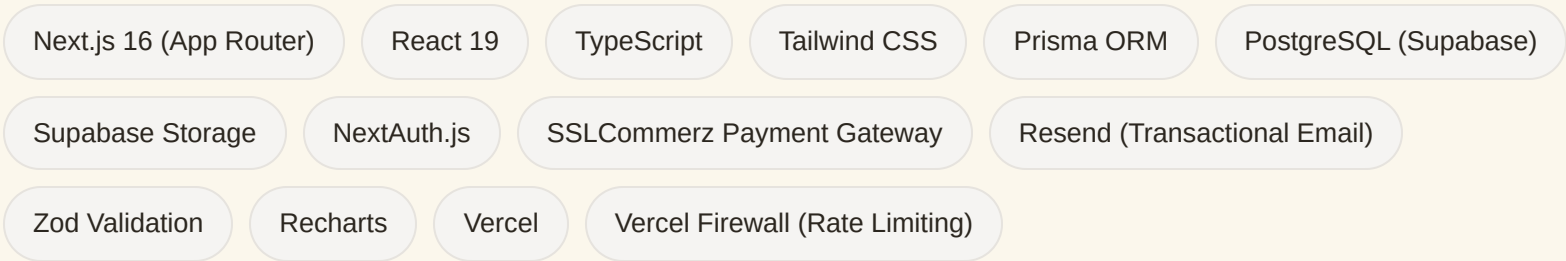
BUSINESS WORKFLOWS

- Menu & category management
- Cart → checkout → payment → tracking pipeline
- Customer accounts & order history
- Admin order management & payment reconciliation
- Sales analytics

TECH STACK

Every layer is real, not mocked

No layer of this stack is a stand-in for "the real thing later" — the payment gateway processes real sandbox transactions with real validation, the email system sends real deliverable email, the auth system issues real sessions.



Four problems that don't show up in a feature list

BUSINESS LOGIC

Protecting business logic from the client

The client submits which items and quantities were selected — never what they cost. Every order's item prices, subtotal, and delivery fee are recalculated server-side from the live Food and Settings tables at the moment the order is created; the request body's own price fields exist only for shape validation and are never used to charge anyone anything. An item that's been deleted or marked unavailable since it was added to the cart blocks the order outright rather than silently accepting stale data.

STATE MANAGEMENT

Modeling the order lifecycle as two independent states

An order has a fulfillment status (Placed → Confirmed → Preparing → Ready → Completed, or Cancelled) and a payment status (Pending, Paid, Failed) — modeled as two separate fields rather than one combined status. That's what makes it possible to represent a real Cash on Delivery order correctly: fully confirmed and being prepared, while payment is still genuinely pending until the driver collects it. Collapsing them into a single enum would have made that state unrepresentable.

ACCESS CONTROL

Authentication & authorization for two different audiences

Customers and restaurant staff share the same login system but not the same permissions — one role field, checked at two independent layers: route middleware that redirects non-admin visitors before an admin page ever renders, and a server-side check on every mutating admin API route underneath it. Guest checkout deliberately doesn't require an account — forcing a signup just to order food is exactly the friction the checkout flow was built to avoid — while a logged-in customer's own orders are still linked automatically via their session, never a client-supplied id.

RESILIENCE

Handling failure in three external integrations

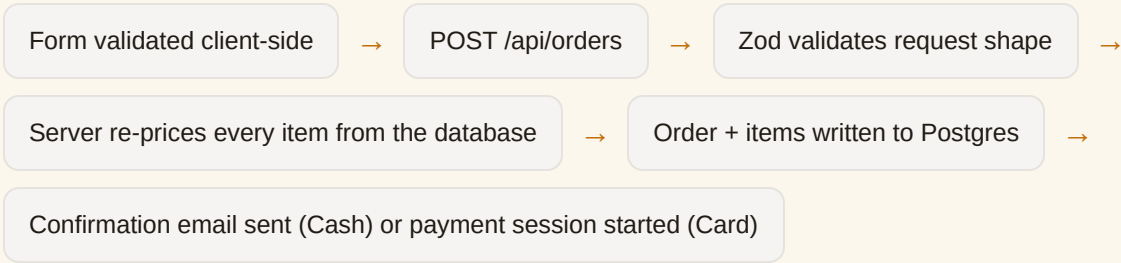
A payment gateway, an email provider, and cloud storage all sit outside this app's control, and each is isolated so a failure in one can't cascade. Every email send is wrapped so a Resend outage can only ever skip an email, never fail a checkout or block an admin action. Payment confirmation doesn't rely solely on the customer's browser redirecting back after paying — a server-to-server webhook runs independently as a fallback, and both paths share the same idempotency guard so a payment can never be double-processed no matter which one fires first, or if both do.

What happens behind the button

Three real user actions, traced from the click to the database and back.

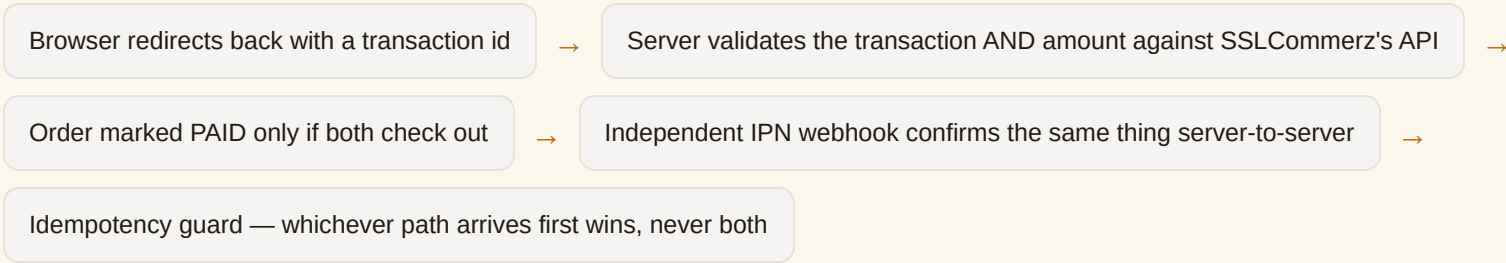
Customer places an order

User sees: Add to cart → Checkout → Place Order



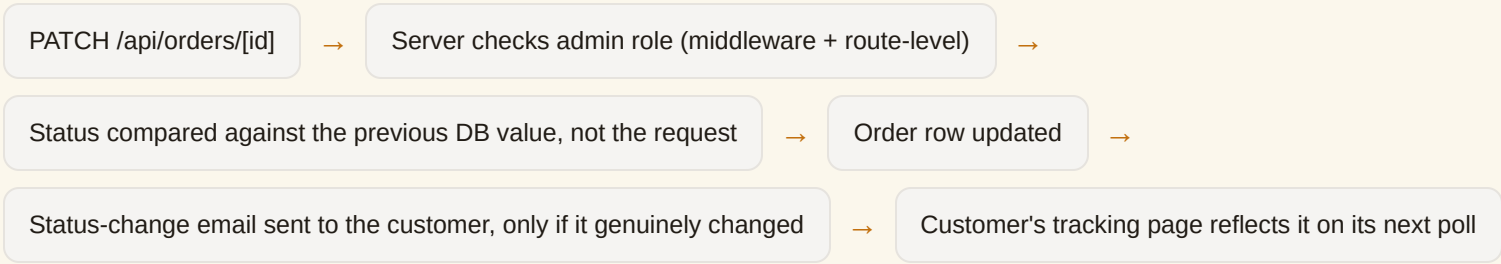
Customer pays online

User sees: Redirected to SSLCommerz → enters card details → redirected back



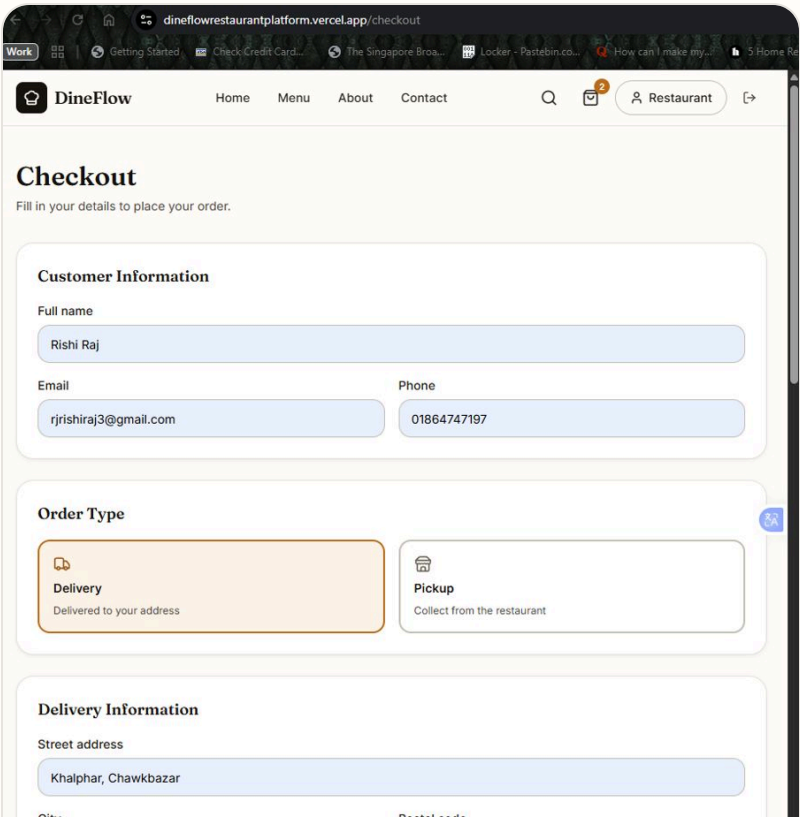
Admin updates an order's status

User sees: Selects "Preparing" from a dropdown on the Orders page



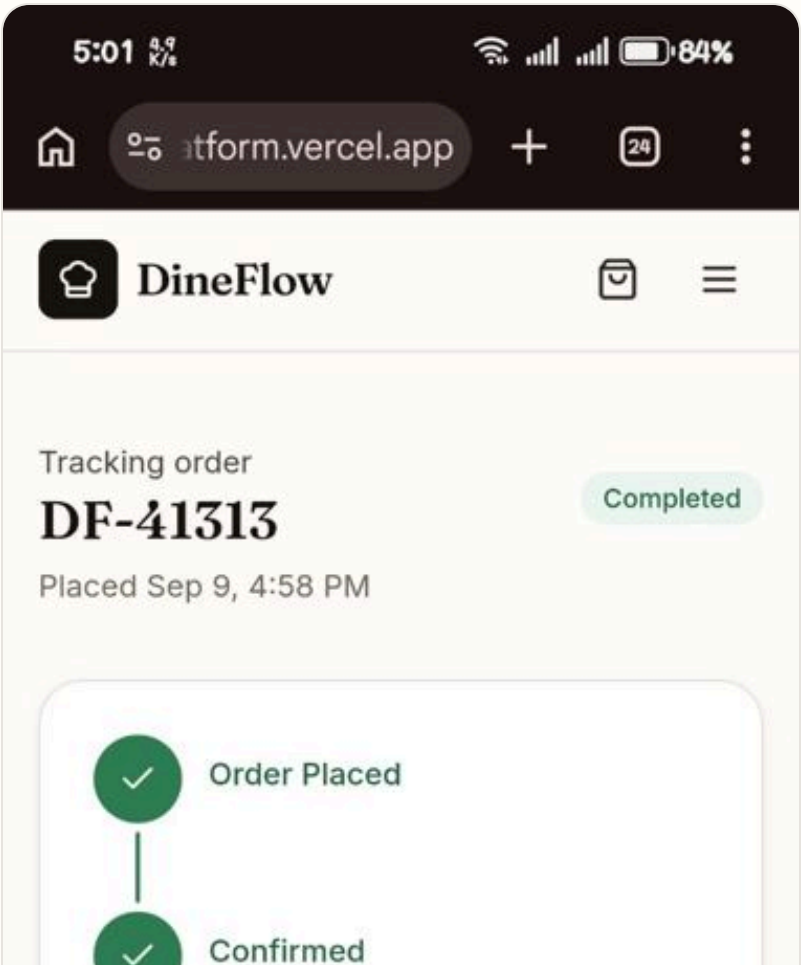
Product walkthrough

Every screenshot below is from the live, deployed site — organized around the workflow it demonstrates, not just what it looks like.



Customer Experience

This is the request that hits the server-side price-validation logic described above — real customer data, recalculated pricing, not a trusted client total.



Ordering Workflow

This view polls the same database row the admin's status dropdown writes to — what the customer sees is never stale, tested on a real phone.



Admin Dashboard

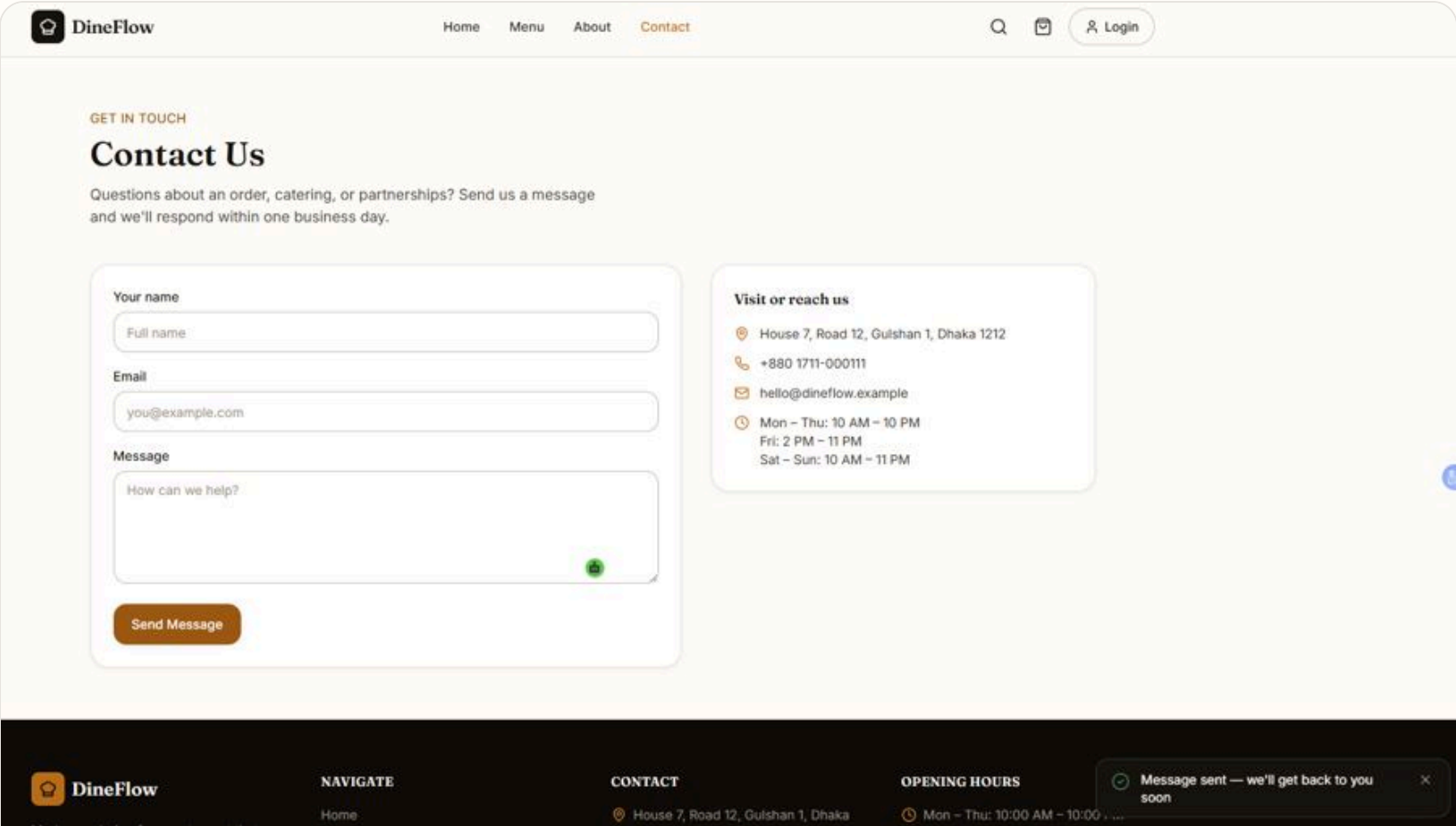
DF-16571	Anttu banua	2 Items	₹760	paid	Sep 2, 9:14 PM	Completed	Set: completed
DF-74044	Riyenshen	3 Items	₹1,260	paid	Sep 2, 9:00 PM	Completed	Set: completed
DF-49461	Riyenshen	2 Items	₹950	pending	Sep 2, 8:54 PM	Completed	Set: completed
DF-75121	Anttu banua	2 Items	₹1,100	paid	Sep 2, 7:33 PM	Completed	Set: completed
DF-95028	Anttu banua	2 Items	₹1,290	pending	Sep 2, 7:28 PM	Cancelled	Set: cancelled
DF-97194	Anttu banua	3 Items	₹1,250	paid	Sep 2, 6:21 PM	Completed	Set: completed
DF-63784	Riyenshen	2 Items	₹950	paid	Sep 1, 9:21 PM	Placed	Set: placed
DF-59925	Riyenshen	6 Items	₹1,890	paid	Sep 1, 9:19 PM	Preparing	Set: preparing
DF-97934	Riyenshen	4 Items	₹1,440	pending	Sep 1, 9:03 PM	Placed	Set: placed
DF-58169	Riyenshen	2 Items	₹900	pending	Sep 1, 9:02 PM	Placed	DF-42367 marked as paid

Business Workflow

The one action in this app that must never be automatic — a human confirming cash actually changed hands, deliberately

Revenue and order trends computed from real orders, not seeded demo numbers — what a restaurant owner would actually check every morning.

walled off from ever touching an online payment's status.

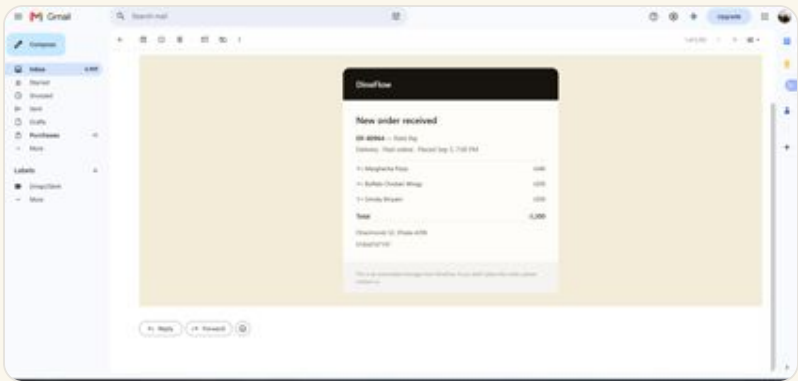


Integration

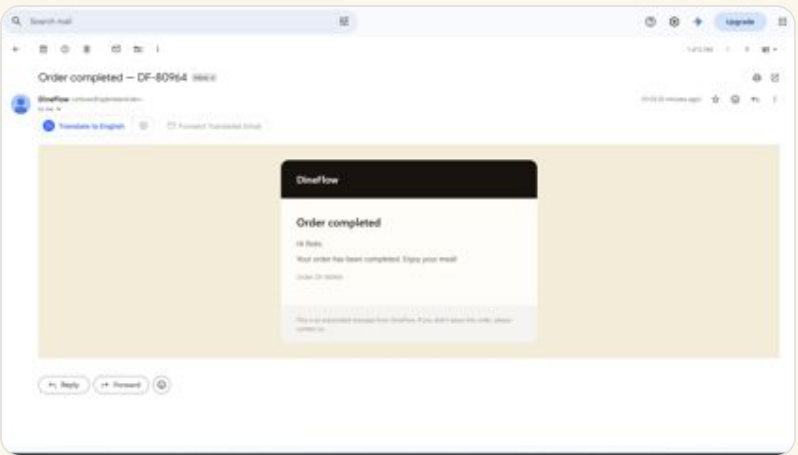
A real email leaving the system through Resend, confirmed with a live success toast — not a mocked network request.

Not claims — proof

Real automated emails, received in a real inbox, generated by a real order placed through the live checkout flow — not a mockup of what an email like this might look like.



Restaurant order alert — sent automatically the moment an order is placed



Customer status update — sent automatically when the admin marks the order complete

What's actually protected, and how

Specific, verifiable measures — not a generic "security is important" statement.

- ✓ Every API route validates its input shape with Zod before touching the database — malformed requests are rejected with a 400, not silently coerced
- ✓ Order pricing is recalculated server-side from the database at checkout — client-submitted prices are structurally present in the request but never trusted for anything financial
- ✓ Admin-only actions are checked at two independent layers: route middleware, and a server-side role check on every mutating API route underneath it
- ✓ Payment confirmation requires SSLCommerz's own server-to-server validation of both the transaction and the amount paid — a client-side "success" redirect is never sufficient on its own
- ✓ Row Level Security is enabled on every database table, closing Supabase's public REST API even though the app itself never uses it — found and fixed after Supabase's own automated scanner flagged it as critical
- ✓ Secrets — database credentials, API keys, the service-role key — live only in server-side environment variables; nothing with write access to the database ships to the browser
- ✓ Order lookups by the public order number require the order's own email as a second factor, since the number alone is a short, guessable code
- ✓ Login, registration, checkout, and the contact form are rate-limited at the edge against scripted abuse
- ✓ The framework itself is kept current — a routine dependency audit surfacing a known vulnerability gets treated as urgent, not backlogged

Built in verified, sequential phases

Each phase was scoped, built, deployed, tested against real infrastructure, and confirmed working before the next one started — not a single big-bang rewrite, and not "done" the moment the features worked.

1 Frontend prototype

Full customer + admin UI on realistic mock data, to validate every flow and get the design system right before touching a database.

2 Database & API layer

PostgreSQL schema via Prisma, 17 REST API routes replacing every mock data source, with Zod validation on every input.

3 Authentication & authorization

NextAuth.js sessions, bcrypt password hashing, route middleware, and server-side role checks on every admin-only endpoint.

4 Payment gateway integration

SSLCommerz sandbox — hosted checkout, server-to-server transaction validation, and a webhook for reliability, with a full success/fail/cancel/retry flow.

5 Media pipeline

Client-side image compression and cloud storage uploads for menu photos, replacing pasted stock-image URLs.

6 Transactional email

Automatic, fail-safe order confirmation, status-update, and restaurant-alert emails — verified via delivery logs, not just "it compiled."

7 Security hardening

Zod validation extended to every remaining route, edge-level rate limiting, Row Level Security across every table, order lookups hardened against enumeration, and every credential rotated off a shared development machine onto dedicated accounts.

8 Framework currency, QA & handover readiness

Migrated off an end-of-life Next.js version using official upgrade tooling, a full manual QA pass with real outside testers, and written documentation so either a non-technical owner or another developer could pick this up cold.

Notable technical decisions

DATA LAYER

Designing for a mock-to-real migration from day one

Every piece of catalog and order state was routed through dedicated Context providers from the very first line of frontend code, shaped to match the eventual database schema. The entire admin CRUD experience was fully built and tested before a database even existed — migrating to real API calls later was a contained, predictable change, not a rewrite.

DATABASE

Serverless-safe Postgres connections

Supabase's connection pooler doesn't support prepared statements the way schema migrations expect. The project uses two distinct connection strings — a pooled connection for the running app (safe under Vercel's serverless concurrency) and a direct connection reserved for Prisma CLI migrations and backups — avoiding a class of connection-exhaustion bugs common in serverless + Postgres deployments.

PAYMENTS

A dual-callback pattern for payment reliability

The checkout flow doesn't rely solely on the customer's browser redirecting back after payment — if their connection drops right after paying, that redirect might never happen. A server-to-server webhook (IPN) runs independently of the customer's browser as a reliable fallback, sharing the same amount-verification and idempotency logic as the redirect path.

MEDIA

Image compression without a native server dependency

Photo uploads are resized and compressed in the browser using the Canvas API before they're ever sent to the server — deliberately avoiding a native image-processing library like Sharp on the server side, which introduces

real deployment risk on serverless platforms. A 12MB phone photo typically becomes a few hundred KB before it ever leaves the customer's — or admin's — browser.

DEPENDENCY MANAGEMENT

Treating framework currency as a security requirement

A routine dependency audit surfaced that the project's Next.js version had reached end-of-life — permanently unpatched, with a critical vulnerability among the findings. The fix meant a real major-version migration (async request APIs, a renamed middleware convention, a coordinated React and ESLint upgrade), done on a branch, verified against a Vercel preview deployment, and merged only after a full manual re-test of every critical flow — the same discipline as any other production change, applied to "just" a dependency update.

DEVELOPMENT PROCESS

Developer + AI, not Developer vs. AI

This project was built working directly with Claude, an AI coding assistant, throughout implementation, debugging, and hardening. That's not a caveat — it's an accurate description of how a growing share of real software gets built now.

What AI accelerated

Implementation speed across routine patterns, systematic debugging (several bugs below were found through structured elimination, not guesswork), generating and reviewing security changes across many files at once, and documentation.

What stayed entirely mine

Every architectural decision, what to build and in what order, testing every change against the real deployed app, deciding which findings actually mattered, and the final judgment call on every line that shipped.

The AI wrote code fast. It didn't decide what was worth building, verify that it actually worked end to end, or notice that a fully-built backend feature had no button wired up to it — that took someone actually using the product like a real user would.

Bugs found before they shipped

Real engineering isn't writing code that looks right — it's catching what's wrong before a user does. These were found during development, code review, and QA — not by a customer in production.

CAUGHT WHILE WRITING THIS CASE STUDY

Rewriting this case study to accurately describe how business logic is protected meant checking the claim against the actual code — and finding it wasn't fully true yet. Item prices, the subtotal, and the total were trusted directly from the checkout request; nothing re-validated them against the real menu prices in the database. A tampered request could have checked out at any price.

Fixed before this case study went live — every price is now recalculated server-side from the database at order creation, and an item deleted or marked unavailable since being added to the cart blocks the order outright.

CAUGHT BY AUTOMATED SECURITY SCAN

Every database table was publicly readable through Supabase's auto-generated API — including password hashes and customer personal data — despite the application itself never using that API. The gap existed purely because Row Level Security wasn't enabled.

Fixed — Row Level Security enabled across all nine tables with zero application code changes required, verified clean on a follow-up scan.

CAUGHT IN QA

A toast notification meant to show "incorrect email or password" never appeared, in any browser, in any mode.

Confirmed step by step: the failed login request returned the correct error, the state-management code was provably updating (verified via direct console logging), and the styling classes involved were valid. The actual cause was a single missing folder in the Tailwind build config's content-scanning list — a component living just outside the paths Tailwind was told to scan, so its styles were silently never generated at all.

Fixed — one-line config change, verified by confirming the component's other styles rendered correctly afterward.

CAUGHT IN QA

Restaurants had no way to mark a Cash on Delivery payment as collected from the Payments page — the backend function for exactly this was already fully built and wired to the Orders page, but the Payments page (the more natural place to look) was purely read-only. A backend being "done" and a feature being reachable turned out to be two different questions, twice — the Orders page and the public demo account surfaced this same gap independently.

Fixed — the same reconciliation action now lives on both pages, restricted only to pending Cash on Delivery orders.

CAUGHT IN REVIEW

Order creation was briefly marking online payments as "paid" immediately at checkout, before the customer had even reached the payment gateway. Combined with how the failure-handling logic worked, this would have permanently prevented an abandoned or declined payment from ever being correctly marked as failed.

Fixed before deployment — every order now starts pending regardless of payment method; only a validated gateway confirmation marks it paid.

CAUGHT IN REVIEW

The original schema required every menu item to belong to a category with no way to remove that link — meaning deleting a category with active menu items would fail at the database level with a constraint error, contradicting what the admin UI already promised the user.

Fixed by making the relationship optional with proper cascading behavior, before it ever reached a real admin's hands.

CAUGHT BY PRODUCTION BUILD

A production build on Vercel caught a genuine type mismatch between seed data and an evolved database schema that local development never surfaced — Next.js's production build runs a full TypeScript check that dev mode skips for speed.

Fixed and redeployed the same day.

Where I'd take this next

Every item on the original build roadmap — security hardening, a full QA pass, and handover documentation — is done. But "done" isn't the same as "nothing left to improve," and knowing the difference matters more than pretending otherwise.

1 Automated testing

The QA pass to date has been entirely manual. A real test suite — especially around the order-pricing and validation logic — would catch regressions before a human has to.

2 A CI pipeline

Vercel's preview deployments catch build failures, but there's no automated lint/test gate before a pull request is even reviewable.

3 Structured observability

Right now, a production issue would need a human to notice it. Real error tracking and logging would surface problems before a user reports them.

4 Verifying the payment integration against live documentation

The SSLCommerz integration is correct against training knowledge and works in sandbox testing, but hasn't been cross-checked line-by-line against their current API docs — I'd do that before any real transaction ever ran through it.

5 More granular admin roles

There's one ADMIN role today. A real restaurant likely wants a kitchen-staff view that can update order status without touching settings or financials.

6 Performance work as it scales

No caching layer exists yet beyond Next.js's own defaults — fine at today's scale, worth revisiting under real load.

Where things stand

AREA	STATUS
Frontend UI (customer + admin)	Live
Database & API layer (PostgreSQL + Prisma)	Live
Authentication & role-based admin access	Live
Payment gateway (SSLCommerz, sandbox)	Live & verified
Image uploads (Supabase Storage)	Live & verified
Transactional email (Resend)	Live & verified
Contact form (real email delivery)	Live
Deployed on Vercel, tested in production	Live
Security hardening (rate limiting, RLS, input validation, access control audit)	Complete
Framework currency (Next.js 16, zero known vulnerabilities)	Complete
Full QA pass across devices, with real outside testers	Complete
Handover documentation (admin guide + technical reference)	Complete

Interested in the full build or working together?

[Live Demo](#) · [GitHub Repository](#) · [LinkedIn](#)